

Método Iterativo de Gradientes Conjugados

COC473 - Álgebra Linear Computacional

Trabalho 2

Igor Valamiel Peixoto Nunes - 124013709
Helena Giovanna Batista da Silva - 119169121
Davi William Santos Araujo - 124104990

Junho 2026

Introdução

Esse projeto tem o objetivo de aplicar conceitos ensinados na disciplina de Álgebra Linear Computacional. Foram usadas matrizes teste e o método de solução estudado foi o Método Iterativo dos Gradientes Conjugados.

Matrizes de Teste

Durante a criação dos códigos, as matrizes usadas para teste foram criadas seguindo o seguinte modelo, sendo que a mesma matriz foi usada para os testes com $\tau = 0.01, 0.05, 0.1, 0.2$:

Considere uma matriz esparsa A de 500×500 construída da seguinte forma. Primeiro, colocamos 1 em cada posição da diagonal e um número aleatório da distribuição uniforme em $[-1, 1]$ em cada posição fora da diagonal (mantendo a simetria $A = A^T$). Em seguida, substituímos por zero cada entrada fora da diagonal com $|a_{ij}| > \tau$, onde τ é um parâmetro. Para τ próximo de zero, o resultado é uma matriz definida positiva bem condicionada cuja densidade de entradas não nulas é aproximadamente τ . Conforme τ aumenta, tanto o número de condicionamento quanto a esparsidade se deterioram.

Código Inicial

Ao iniciar o programa, serão pedidos o tamanho da matriz quadrada (que representa o sistema), a matriz em si, e o vetor. Caso deseje apenas realizar um teste com um modelo padrão, basta responder tal.

- A matriz: Digite uma linha, separando os números por um espaço, e aperte enter para passar de linha.
- O vetor: Digite os N números separando-os por um espaço e aperte enter.

O output é a solução do sistema, o tempo de execução do método e um gráfico mostrando as iterações para cada valor de τ .

Exemplo de exibição no terminal:

```
=====
Boas Vindas ao solucionador de Equações Lineares por Gradientes Conjugados!

Para o bom funcionamento do programa, siga as seguintes regras:
1) Insira o valor inteiro N da matriz NxN;
2) Aparecerão N linhas de input, insira os valores de cada linha separando com um espaço;
3) Insira os valores do vetor em uma única linha, separando os números com um espaço.
=====
Você quer inserir sua matriz? [0 : não - 1 : sim] 1
N = 10
Linha 1: 4 1 0 0 0 0 0 0 0 0
Linha 2: 1 4 1 0 0 0 0 0 0 0
```

```

Linha 3: 0 1 4 1 0 0 0 0 0 0
Linha 4: 0 0 1 4 1 0 0 0 0 0
Linha 5: 0 0 0 1 4 1 0 0 0 0
Linha 6: 0 0 0 0 1 4 1 0 0 0
Linha 7: 0 0 0 0 0 1 4 1 0 0
Linha 8: 0 0 0 0 0 0 1 4 1 0
Linha 9: 0 0 0 0 0 0 0 1 4 1
Linha 10: 0 0 0 0 0 0 0 0 1 4
Vetor: 5 6 6 6 6 6 6 6 6 5

```

Realizando Método Iterativo de Gradientes Conjugados:

```
tau = 0.01
```

```
-----
Matriz simétrica, e positiva definida!
```

```

x1: 1.25
x2: 1.5
x3: 1.5
x4: 1.5
x5: 1.5
x6: 1.5
x7: 1.5
x8: 1.5
x9: 1.5
x10: 1.25

```

```

it [0] :      r ~> 18.384776310850235
it [1] :      r ~> 0.0

```

```
tau = 0.05
```

```
-----
Matriz simétrica, e positiva definida!
```

```

x1: 1.2359472087873205
x2: 1.4985582289240171
x3: 1.526571236277418
x4: 1.51671699788683
x5: 1.5
x6: 1.5
x7: 1.5
x8: 1.5073197824170517
x9: 1.5
x10: 1.2587651401270115

```

```

it [0] :      r ~> 18.384776310850235
it [1] :      r ~> 0.13255449325671628
it [2] :      r ~> 0.001317244786697338
it [3] :      r ~> 8.775518493304365e-06
it [4] :      r ~> 5.329196666465397e-08
it [5] :      r ~> 9.257742101176758e-11
it [6] :      r ~> 1.501048063626117e-13
it [7] :      r ~> 1.3310219185103205e-26

```

```
tau = 0.1
```

```
-----
Matriz simétrica, e positiva definida!
```

```

x1: 1.205520746580821
x2: 1.4988412918319292
x3: 1.5262309009980763
x4: 1.4854662505890397
x5: 1.5000000000000002
x6: 1.4423922835456466
x7: 1.4738903926666926
x8: 1.5071374585411303

```

```

x9: 1.5000000000000002
x10: 1.2274113476033304
~~~~~

it [0] :    r ~> 18.384776310850235
it [1] :    r ~> 0.30938645524231995
it [2] :    r ~> 0.004615709089005751
it [3] :    r ~> 7.586219304594934e-05
it [4] :    r ~> 1.5556333226667665e-06
it [5] :    r ~> 1.0492236044580242e-08
it [6] :    r ~> 6.459684141866027e-11
it [7] :    r ~> 7.679176622000388e-14
it [8] :    r ~> 1.7768719859712237e-26
~~~~~

tau = 0.2

-----
Matriz simétrica, e positiva definida!
x1: 1.2081833048739075
x2: 1.4370084690682485
x3: 1.5258432251926464
x4: 1.4876010782916982
x5: 1.5000000000000002
x6: 1.3436506315235295
x7: 1.474671892163649
x8: 1.506923823759564
x9: 1.5000000000000002
x10: 1.190673094956901
~~~~~

it [0] :    r ~> 18.384776310850235
it [1] :    r ~> 0.6461487369615487
it [2] :    r ~> 0.020888240838940107
it [3] :    r ~> 0.00037975921825013646
it [4] :    r ~> 6.746612512965051e-06
it [5] :    r ~> 5.704702862675616e-08
it [6] :    r ~> 4.07788530469613e-10
it [7] :    r ~> 3.0538182118378876e-13
it [8] :    r ~> 5.321233537667633e-24
~~~~~

```

Código (main.py):

```

from CG_estruturas import CG_Matriz, teste_de_aplicacao, CG_transpose
from estruturas import Matriz
from random import randint

def calling_CG_method(n, A, b, r_min, it_max):
    x = Matriz(n, 1)
    v = b - (A * x)
    r = b - (A * x)

    Beta = 0
    it = 0
    it_list = [0]
    r_list = [r.module()]

    r_old = CG_transpose(r) * r

    for it in range(1, it_max + 1):

        Av = A * v
        vAv = CG_transpose(v) * Av

        # passo
        ai = r_old/vAv

```

```

# atualizacao de x (solucao)
x += v.multiplicar(ai)

# atualizacao de r (residuo)
r -= Av.multiplicar(ai)

r_mod = r.module() # modulo do vetor de residuo
it_list.append(it)
r_list.append(r_mod)

if r_mod <= r_min:
    break

r_new = CG_transpose(r) * r
Beta = r_new/r_old # atualizacao de Beta (fator correcao)
r_old = r_new
v = r + (v.multiplicar(Beta)) # atualizacao do v (vetor de direcao)

return x, it_list, r_list

def make_graph(dados_dos_testes):
    import matplotlib.pyplot as plt
    plt.figure(figsize=(10, 6))

    for tau, (itList, rList) in dados_dos_testes.items():
        plt.plot(itList, rList, label=f'tau = {tau}', marker='o', markersize=3)

    plt.xlabel('Iterações')
    plt.ylabel('Módulo do Resíduo (r)')
    plt.title('Convergência do Gradiente Conjugado para diferentes valores de Tau')
    plt.yscale('log')
    plt.grid(True, which="both", ls="--")
    plt.legend()
    plt.show()

def CG_method(n=500, A=None, b=None, r_min=1e-16, it_max=20):

    if A is None:
        A = CG_Matriz(500, 500)

    if b is None:
        random_list = [randint(-10, 10) for _ in range(n)]
        b = Matriz(n, 1, random_list)

    data_dict = {}

    for tau in [0.01, 0.05, 0.1, 0.2]:
        print(f"tau = {tau}")
        print('_'*50)

        # testando matriz
        A_filtrada = A.filtro_de_corte(tau)
        teste = teste_de_aplicacao(A_filtrada, n)
        if not teste[0]: raise teste[1]
        else: print(teste[1])

        # fazendo o metodo
        M, itList, rList = calling_CG_method(n, A_filtrada, b, r_min, it_max)
        data_dict[tau] = (itList, rList)

    # printando os resultados
    for i in range(1, n+1):

```

```

        print(f"x{i}: {M.mat[i-1][0]}")

    print('~'*50)

    for i in range(len(itList)):
        print(f"it [{itList[i]}] :      r ~> {rList[i]}")

    print('~'*50)

make_graph(data_dict)

```

Método Iterativo de Gradientes Conjugados

Código (CG_method.py)

```

from CG_estruturas import CG_Matriz, teste_de_aplicacao, CG_transpose
from estruturas import Matriz
from random import randint

def calling.CG_method(n, A, b, r_min, it_max):
    x = Matriz(n, 1)
    v = b - (A * x)
    r = b - (A * x)

    Beta = 0
    it = 0
    it_list = [0]
    r_list = [r.module()]

    r_old = CG_transpose(r) * r

    for it in range(1, it_max + 1):

        Av = A * v
        vAv = CG_transpose(v) * Av

        # passo
        ai = r_old/vAv

        # atualizacao de x (solucao)
        x += v.multiplicar(ai)

        # atualizacao de r (residuo)
        r -= Av.multiplicar(ai)

        r_mod = r.module() # modulo do vetor de residuo
        it_list.append(it)
        r_list.append(r_mod)

        if r_mod <= r_min:
            break

        r_new = CG_transpose(r) * r
        Beta = r_new/r_old # atualizacao de Beta (fator correcao)
        r_old = r_new
        v = r + (v.multiplicar(Beta)) # atualizacao do v (vetor de direcao)

    return x, it_list, r_list

# -----

```

```

def make_graph(dados_dos_testes):
    import matplotlib.pyplot as plt
    plt.figure(figsize=(10, 6))

    for tau, (itList, rList) in dados_dos_testes.items():
        plt.plot(itList, rList, label=f'tau = {tau}', marker='o', markersize=3)

    plt.xlabel('Iterações')
    plt.ylabel('Módulo do Resíduo (r)')
    plt.title('Convergência do Gradiente Conjugado para diferentes valores de Tau')
    plt.yscale('log')
    plt.grid(True, which="both", ls="--")
    plt.legend()
    plt.show()

# -----

def CG_method(n=500, A=None, b=None, r_min=1e-16, it_max=20):

    if A is None:
        A = CG_Matriz(500, 500)

    if b is None:
        random_list = [randint(-10, 10) for _ in range(n)]
        b = Matriz(n, 1, random_list)

    data_dict = {}

    for tau in [0.01, 0.05, 0.1, 0.2]:
        print(f"tau = {tau}")
        print('~'*50)

        # testando matriz
        A_filtrada = A.filtro_de_corte(tau)
        teste = teste_de_aplicacao(A_filtrada, n)
        if not teste[0]: raise teste[1]
        else: print(teste[1])

        # fazendo o metodo
        M, itList, rList = calling_CG_method(n, A_filtrada, b, r_min, it_max)
        data_dict[tau] = (itList, rList)

        # printando os resultados
        for i in range(1, n+1):
            print(f"x{i}: {M.mat[i-1][0]}")

        print('~'*50)

        for i in range(len(itList)):
            print(f"it [{itList[i]}] : r ~> {rList[i]}")

        print('~'*50)

    make_graph(data_dict)

```

Gráficos

Exemplos de Gráfico:

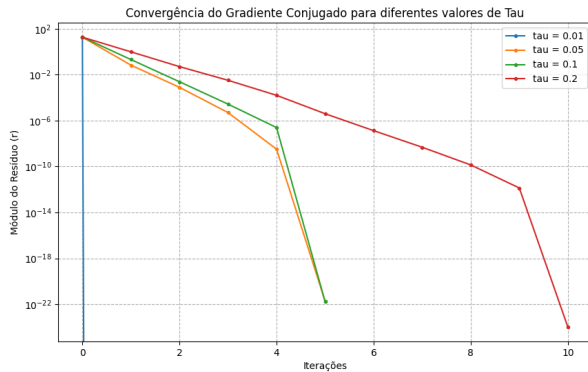


Figure 1: Matriz Exemplo 10x10

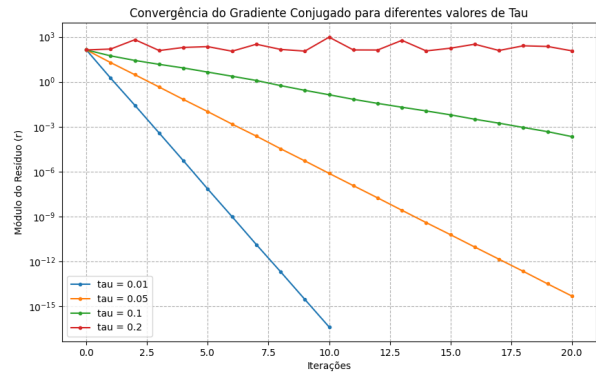


Figure 2: Matriz Exemplo 500x500

Códigos Auxiliares (também disponíveis no github)

CG_estruturas.py

Arquivo usado para criar a classe Matriz, na qual todos os objetos usados para os métodos pertencem.

CG_method.py

Arquivo que contém as funções da biblioteca principal.

Conclusão

- O Método dos Gradientes Conjugados converge rapidamente para valores baixos de τ (como 0.01), pois a matriz preserva as suas propriedades matemáticas essenciais (simétrica e definida positiva).
- O aumento do valor de τ (como 0.1 e 0.2) torna a matriz mais esparsa, mas deteriora severamente o seu número de condicionamento, resultando em estagnação e grande dificuldade de atingir o critério de parada.
- A eficiência computacional do método iterativo não é garantida apenas pelo aumento da esparsidade (quantidade de zeros), mas depende fundamentalmente da preservação da estabilidade estrutural e espectral do sistema.

Por fim, o trabalho evidenciou a extrema sensibilidade do Gradiente Conjugado às propriedades da matriz, reforçando a importância de uma análise prévia do condicionamento do sistema antes da aplicação prática de métodos numéricos iterativos.

Uso de Ferramentas de IA e Link para o código

Para realizar esse trabalho, foi utilizado o Gemini-pro para estudantes, para entender melhor o trabalho, gerar testes e ajudar no debug dos códigos:

- Gemini-Estudante: Não é possível compartilhar as mensagens.

Este link direciona ao repositório no github que contém todos os códigos implementados:

<https://github.com/igorvalamiel/AlgLinComp/tree/main/trab2>